

CURRICULUM DOCUMENT

Automation with Generative AI

#	Module	Hours
01	Generative AI Foundations & LLM Architecture	50 Hours
02	Automation Platforms & Workflow Design	45 Hours
03	Generative AI Models & Tool Exploration	45 Hours
04	Agentic AI Frameworks	35 Hours
05	Python for Generative AI — Integration and Implementation	45 Hours
Total Program Hours		220 Hours

MODULE 01

Generative AI Foundations & LLM Architecture**50**

TOTAL HOURS

This is the most important module in the curriculum. It carries the most hours deliberately. Everything that comes after — prompt engineering, model selection, RAG pipelines, agent design, integration architecture — is better understood by someone who knows how Large Language Models actually work than by someone who only knows how to use them.

The objective is not to produce machine learning researchers. The mathematics of attention mechanisms and the backpropagation algorithms behind training are not covered in detail. The objective is to produce practitioners who understand what is happening when they send a prompt and receive a response — at a level deep enough to make good decisions about how to construct prompts, choose models, design retrieval systems, and debug unexpected behaviour.

A common mistake in AI education is to teach people how to use tools without teaching them why those tools behave the way they do. That approach produces practitioners who can follow tutorials but cannot adapt when things go wrong. This module takes the opposite approach — it builds understanding first, and that understanding then makes every tool in every subsequent module easier to use correctly.

How LLMs Work

At its core, a Large Language Model does one thing: given a sequence of tokens, it predicts the most probable next token. This simple description conceals enormous complexity, but it is also genuinely explanatory — understanding it resolves many puzzles about model behaviour, including why models hallucinate, why they are sensitive to phrasing, and why context window limits matter so much.

The Transformer architecture, introduced in the landmark 2017 paper, is the mechanism behind every major language model. Its key innovation — the attention mechanism — allows the model to weigh the relevance of every other token in the context window when predicting the next token. This is what enables models to maintain coherence across long documents and to understand references, pronouns, and implicit context.

Training proceeds in stages. Pre-training exposes the model to vast quantities of text, teaching it the statistical patterns of language. Instruction tuning fine-tunes the pre-trained model on human demonstrations of desired behaviour, teaching it to follow instructions. Reinforcement Learning from Human Feedback (RLHF) further shapes the model to produce responses that human raters prefer, which is how safety and helpfulness properties are instilled.

Scale has produced surprises that were not predicted from first principles. As models grew from millions to billions to hundreds of billions of parameters, they began exhibiting capabilities that smaller models did not have at all — multi-step reasoning, code generation, analogy-making across distant domains. These emergent capabilities were not explicitly trained for; they arose from the scale of the model and the breadth of its training data.

Understanding the difference between a base model and an instruction-tuned model has direct practical consequences. A base model, given a prompt, will continue it — not answer it. It has learned to predict the next token, not to respond helpfully. An instruction-tuned model has been trained to recognise a question as a request for a response and to produce one. All the models participants interact with in this curriculum are instruction-tuned, but understanding that this is a distinct training step — not an inherent property of language models — matters for understanding why models sometimes still produce unhelpful continuations rather than responses.

- Language models as next-token predictors — the foundational mechanism behind all LLMs
- The Transformer architecture — the structural innovation that made modern LLMs possible
- Attention mechanism — how models determine which parts of the context are relevant to each prediction
- Self-attention and multi-head attention — processing context in parallel across multiple representation subspaces
- Scale and emergent capability — how increasing parameters, data, and compute produces qualitatively new abilities
- Pre-training — learning language patterns from massive text corpora
- Instruction tuning — teaching the model to follow instructions by fine-tuning on demonstrations
- RLHF — shaping model behaviour through human preference feedback
- Base models vs instruction-tuned models vs chat models — understanding the distinctions

Tokenization, Embeddings, and Model Parameters

Before a language model can process text, that text must be converted into numbers. The process of conversion is tokenization, and understanding it has direct practical consequences: it determines what fits in a context window, why some inputs cost more than others, and why models sometimes behave strangely at word boundaries.

Embeddings are the representation of meaning in numerical form. When a model processes a token, it maps that token to a point in a very high-dimensional vector space. Tokens with similar meanings cluster near each other in this space. This is how models understand that 'king' and 'monarch' are semantically related, and it is also the foundation of semantic search and retrieval systems.

Model parameters — temperature, top-p, context window, max tokens — are the controls available to practitioners when calling a model. Understanding what each parameter does mechanistically, not just descriptively, allows practitioners to configure models appropriately for different tasks rather than defaulting to whatever the examples show.

The context window is the most consequential parameter for system design. A model with a 200,000 token context window can hold an entire book — or an entire codebase — in a single call. A model with a 4,000 token context window cannot hold a long email thread. The architectural decisions that follow from context window size are significant: whether RAG is needed, how conversation history must be managed, whether a document can be processed in a single call or must be split across multiple calls.

Temperature is frequently misunderstood. It is not simply a creativity dial — it is a control over the probability distribution from which the next token is sampled. At temperature zero, the model always selects the single most probable token, producing deterministic and often repetitive output. At high temperatures, low-probability tokens are sampled frequently, producing varied and sometimes incoherent output. For extraction tasks, classification, and structured output, temperature should be low. For creative writing and brainstorming, moderate temperature is appropriate.

- Tokenization — how text is split into subword units before a model processes it
- Byte-pair encoding (BPE) — the dominant tokenization algorithm and how it handles uncommon words
- Token counts and cost — the direct relationship between tokenization and API pricing
- Context window — the maximum number of tokens a model can process at once, including both input and output
- Embeddings — vector representations of tokens that encode semantic relationships
- Vector space — how semantic similarity is captured as geometric proximity
- Vector databases — systems for storing and querying embeddings at scale
- Temperature — controlling the probability distribution over possible next tokens
- Top-p (nucleus sampling) — sampling only from the tokens whose cumulative probability exceeds p

- Max tokens — capping output length and managing truncation
- Stop sequences — strings that terminate generation when encountered in output
- Frequency and presence penalties — reducing repetition in extended outputs

Prompt Engineering

Prompt engineering is a systematic discipline, not an intuitive art. It has known patterns, documented failure modes, and reproducible debugging approaches. This session treats it that way — teaching prompting as a craft with principles, not as a collection of tricks to try when the obvious thing does not work.

The most important skill in prompt engineering is not knowing the patterns — it is knowing why they work. Zero-shot prompting works for tasks the model already understands well. Few-shot prompting works by demonstrating the pattern when the task has an unusual format. Chain-of-thought prompting works by making the model externalise intermediate reasoning steps that it would otherwise skip. Understanding the mechanism behind each pattern allows practitioners to apply them appropriately rather than mechanically.

Structured output prompting — reliably eliciting JSON, tables, and formatted lists from models — deserves particular emphasis because it is the foundation of every AI integration that needs to pass model output to a downstream system. A model that returns free-form prose cannot be reliably parsed by an automation workflow. A model that returns structured JSON can. Techniques for achieving reliable structured output are covered in depth.

System prompts are one of the most underused tools in prompt engineering. Most practitioners focus entirely on the user message and treat the system prompt as a label. In reality, the system prompt is where the most durable and reusable prompt engineering happens — defining the model's role, its output format, its constraints, and its persona in a way that applies consistently across every user message in a conversation. A well-designed system prompt reduces the complexity of every subsequent user prompt.

Prompt versioning is introduced here as a professional discipline. Prompts are code. They should be tracked in version control, tested against a consistent evaluation set before deployment, and changed deliberately rather than casually. The most common failure mode in AI-powered applications is a prompt change that improves one case while silently breaking three others — a failure that is only detected after it has affected users. Treating prompts with the same rigour as application code prevents this category of failure.

- Zero-shot prompting — giving the model a task without examples and when it is appropriate
- Few-shot prompting — providing input-output examples to demonstrate the expected pattern
- Chain-of-thought prompting — instructing models to reason step by step before providing an answer
- System prompts — defining the model's role, tone, constraints, and output format at the conversation level
- Structured output prompting — techniques for reliably getting JSON, tables, and formatted content
- Prompt chaining — breaking complex tasks into sequences of simpler, more reliable prompts
- Negative prompting — specifying what not to do as explicitly as what to do
- Role and persona definition — how explicit role assignment affects model behaviour
- Common prompt failure patterns — vague instructions, conflicting constraints, ambiguous format requests
- Debugging prompts — identifying why a prompt fails and iterating systematically toward a fix
- Prompt versioning — tracking prompt changes and measuring the impact of modifications
- Prompt injection — understanding how malicious inputs can subvert prompt instructions

Retrieval Augmented Generation (RAG)

Every language model has a knowledge cutoff — a point in time beyond which it has no information. Every language model also has a tendency to hallucinate — to produce plausible-sounding information that is factually incorrect. RAG is the primary architectural approach for addressing both limitations in production AI systems.

The core idea is straightforward: rather than relying on what the model memorised during training, retrieve relevant information from an external source at query time and provide it to the model as context alongside the question. The model then generates a response grounded in the retrieved information rather than its training data alone.

Building a RAG pipeline involves several distinct steps, each with its own set of design decisions. How documents are chunked affects retrieval quality. How embeddings are generated affects what kinds of similarity can be detected. How retrieval is configured affects whether the most relevant chunks are actually returned. How retrieved context is presented in the prompt affects how well the model uses it. This session covers each step and the design decisions it involves.

- The knowledge cutoff problem — why models cannot know what happened after their training data was collected
- The hallucination problem — why models generate plausible-sounding false information and when it is most likely
- RAG architecture — the end-to-end pipeline from document ingestion to grounded response generation
- Chunking strategies — fixed-size, sentence-level, semantic, hierarchical, and recursive chunking approaches
- Embedding documents — converting chunks into vectors for similarity-based retrieval
- Indexing — storing embeddings in a vector database for efficient querying
- Retrieval — finding the most relevant chunks for a given query using embedding similarity
- Hybrid retrieval — combining vector similarity search with keyword search for better recall
- Reranking — using a secondary model to reorder retrieved chunks by relevance
- Augmentation — constructing the final prompt by combining retrieved context with the user query
- Context window management — handling cases where retrieved content exceeds available context
- RAG evaluation — measuring retrieval precision, answer faithfulness, and answer relevance
- Common RAG failure modes — retrieval misses, context overflow, conflicting retrieved chunks
- When to use RAG vs fine-tuning — making the right architectural choice for a given use case

Fine-Tuning, Multimodal AI, Evaluation, and Ethics

This session covers four interconnected topics that complete the foundational picture. Fine-tuning is presented as an architectural option with specific trade-offs, not as a universal improvement over prompting. Multimodal AI extends the understanding of language models to cover the full range of input and output modalities now available. Evaluation introduces the discipline of measuring whether AI systems are actually performing well. Ethics grounds the technical content in the real-world consequences of deploying AI systems.

- Fine-tuning — adjusting model weights on a task-specific dataset to specialise behaviour
- When fine-tuning is justified — consistent format requirements, large labelled datasets, exhausted prompt engineering
- When fine-tuning is not appropriate — small datasets, changing requirements, unclear evaluation criteria
- Cost of fine-tuning — compute, data preparation, evaluation overhead, and ongoing maintenance burden
- Text-to-image — how diffusion models generate images from text prompts
- Image-to-text — vision encoders and how models understand and describe visual content
- Document understanding — extracting structured information from PDFs, forms, and screenshots using vision models
- Text-to-speech — neural voice synthesis and the current quality ceiling
- Speech-to-text — transcription accuracy, language coverage, and latency trade-offs
- Text-to-video — capabilities, limitations, and the gap between research demos and production use
- Natively multimodal models — models designed from the ground up to handle mixed input types

- Model evaluation — accuracy, consistency, hallucination rate, latency, cost, and format compliance
- Hallucination — why it happens, how to detect it, and how to design systems that mitigate its effects
- Bias — how training data biases surface in model outputs and the difficulty of measuring them
- Data privacy — what information should and should not be placed in model context
- Responsible deployment — output filtering, guardrails, and human-in-the-loop checkpoints

Key Concepts	
Transformer	The neural network architecture behind every major language model, built on attention mechanisms.
Attention Mechanism	The component of a Transformer that allows the model to weigh the relevance of each token when predicting the next.
Embedding	A numerical vector representation of a token or piece of text that encodes its semantic meaning.
Vector Database	A database optimised for storing and querying high-dimensional vectors, used in RAG and semantic search systems.
RAG	Retrieval Augmented Generation — grounding model responses with information retrieved from external sources at query time.
Hallucination	When a model generates information that sounds plausible but is factually incorrect.
Context Window	The maximum amount of text (measured in tokens) that a model can process in a single call.
Chunking	The process of splitting documents into smaller pieces for embedding and retrieval in a RAG pipeline.
RLHF	Reinforcement Learning from Human Feedback — a training technique that shapes model behaviour using human preference ratings.
Temperature	A parameter controlling the randomness of model output by adjusting the token probability distribution.
Instruction Tuning	Fine-tuning a base model on instruction-response pairs to teach it to follow natural language directions.
Semantic Chunking	A chunking strategy that splits documents on topic boundaries rather than fixed character counts.
Prompt Caching	A technique that reuses computation for repeated prompt prefixes, reducing latency and cost.

Practice Exercises

1. Write a prompt for a model to extract structured JSON from an unstructured customer complaint — name, product, issue, urgency level. Test it on five different complaints and evaluate consistency.
2. Design a simple RAG pipeline using LlamaIndex: ingest a 10-page PDF, chunk it, embed it, and build a query interface that retrieves relevant sections and generates an answer.
3. Compare the same complex reasoning task (multi-step maths problem or logical puzzle) across GPT-4o, Claude, and Gemini. Document how each model approaches the problem differently.
4. Create a prompt evaluation rubric with five criteria and use it to systematically compare five different prompts for the same task. Document what changed between prompts and what effect each change had.

5. Build a prompt that reliably extracts a structured JSON object with six fields from a variety of unstructured invoice texts. Test it on ten different invoice formats and measure field extraction accuracy.
6. Implement a semantic chunking function in Python that splits a document on paragraph boundaries, embeds each chunk using the OpenAI embeddings API, and stores the results in a local ChromaDB instance.

MODULE 02

Automation Platforms & Workflow Design**45**

TOTAL HOURS

Automation is a discipline with its own principles, vocabulary, and failure modes that are entirely independent of AI. A workflow in Make or n8n that moves data between a CRM and a spreadsheet involves no AI at all. Conversely, calling Claude to summarise a document is Generative AI work with no automation involved. The intersection — where Generative AI is embedded inside automation pipelines — is powerful, but it can only be understood properly by someone who understands each discipline on its own terms first.

The module begins with a session on automation thinking — the conceptual framework for approaching automation problems — before any platform is introduced. Participants who understand the principles can work in any automation platform. Participants who only know how to click through one platform's interface are helpless when the platform changes or when they encounter a problem the interface does not expose a solution for.

Two platforms are covered in depth. Make.com is the right tool when visual orchestration and breadth of native integrations matter — it excels at connecting business tools quickly with minimal configuration. n8n is the right tool when complexity, control, and the ability to self-host matter — it handles advanced branching, custom logic, and large data volumes that Make cannot. Both are production-grade platforms used in real organisations.

Automation Thinking

Most people approaching automation platforms for the first time try to figure out the tool before they have understood the problem. They open Make or n8n, look for a trigger that resembles their starting point, and start connecting modules without a clear model of what they are building. The result is workflows that work for the happy path but break on the first edge case — because the edge cases were never considered during design.

This session teaches participants to think about automation before they touch a tool. What is the trigger? What is the outcome? What data needs to move between which systems? What can go wrong at each step? What should the workflow do when something does go wrong? A participant who can answer these questions before opening a platform will build workflows that are faster to construct, easier to debug, and more reliable in production.

A particular emphasis is placed on designing for failure. In production automation, workflows run unattended. When something goes wrong — an API is down, a record has an unexpected format, a required field is missing — nobody is watching. A workflow designed for the happy path will fail silently or corrupt data. A workflow designed for failure will log the problem, alert the right person, and either retry or stop cleanly.

The hidden cost of automation is a concept introduced here and revisited throughout the module. A workflow that runs reliably for six months and then breaks because an upstream API changed its response format has a real cost — the time to diagnose and fix it, the data that was missed while it was broken, and the trust that was lost with whoever depended on it. Designing for maintainability — using clear naming, documenting data flows, building in validation checkpoints — reduces this cost significantly.

Participants work through a structured exercise in this session: given a manual business process described in plain language, they identify every step, classify it as trigger, action, condition, or transformation, map the data that flows between steps, and identify every place where the process can fail. This exercise is completed before any platform is introduced — because the quality of the workflow design is determined before the first module is placed on a canvas.

- What automation is — the systematic, unattended execution of a defined process

- What automation is not — it does not handle ambiguity, exceptions, or judgement calls without being told how
- Triggers — the initiating event: scheduled time, incoming webhook, new record, form submission, email arrival
- Actions — the operations the workflow performs: create, read, update, delete, send, transform
- Conditions — branching logic that routes workflow execution based on data values
- Loops — repeating an action over each item in a collection
- Data mapping — taking a field from one system and writing it to the corresponding field in another
- Data transformation — reshaping, cleaning, filtering, and enriching data as it moves through a workflow
- Identifying automation candidates — repetitive, rule-based, high-volume, consistent, low-variation tasks
- Designing for failure — what the workflow does at each step if something goes wrong
- Idempotency — designing workflows that can be safely re-run without duplicating effects
- Retry logic — attempting failed steps again with appropriate delays
- Dead letter handling — what to do with records that have failed all retries
- Monitoring — knowing when a production workflow is failing or degraded
- The hidden cost of automation — maintenance burden when connected systems change

APIs, Webhooks, and Data Handling

Automation platforms connect systems. The mechanism for connection is almost always an API or a webhook. Native connectors in Make and n8n are convenient wrappers around API calls — they abstract the HTTP details but they cannot cover every API in existence. A practitioner who understands the underlying API communication can connect to any system, not just the ones with native connectors.

Data handling is the skill that determines whether a workflow actually produces correct results. Data rarely arrives from one system in exactly the format the next system expects. Dates are formatted differently. Names are split or combined differently. Numbers need rounding or currency conversion. Required fields are sometimes missing. The ability to reliably transform, validate, and route data is what separates a workflow that works from one that works most of the time.

Pagination is a particularly common source of silent data loss in automation workflows. An API that returns results in pages will return the first page on the first call and require subsequent calls with a cursor or page number to retrieve the rest. A workflow that does not implement pagination will process only the first page of results and silently miss everything else — with no error, no warning, and no indication that data has been lost.

Webhook security is introduced here because it is routinely neglected and routinely exploited. A webhook endpoint that accepts any POST request will process data from any sender — including attackers who have discovered the URL. Validating the signature that providers include in the request headers — a hash of the request body signed with a shared secret — ensures that only the genuine provider can trigger the workflow. Every webhook integration built in this program includes signature validation as a non-optional step.

- REST APIs — resources, endpoints, HTTP methods, and the request-response cycle
- Authentication — API keys in headers, OAuth 2.0 token flows, and how platforms manage credentials
- Request construction — base URL, path parameters, query strings, request headers, and body
- Response handling — status codes, response body parsing, and error detection
- Pagination — iterating through multiple pages of API results automatically
- Webhooks — external systems pushing event data to your workflow's endpoint URL
- Polling vs webhooks — comparing the approaches and choosing appropriately
- Webhook security — signature validation to prevent spoofed requests
- Connecting via HTTP modules — calling any API without a native connector
- JSON navigation — accessing nested fields, handling arrays, and managing missing keys

- String manipulation — trimming, splitting, replacing, and formatting text fields
- Date and time handling — parsing, formatting, timezone conversion, and date arithmetic
- Number operations — arithmetic, rounding, formatting for different system expectations
- Array operations — filtering, mapping, aggregating, and iterating over collections
- Data validation — checking that required fields exist and have expected types
- Handling null and missing data — defaults, fallbacks, and workflow branching on missing values

Make.com

Make is the platform for participants who need to build automation workflows efficiently and visually. Its strength is the breadth and quality of its native integrations — over 1,500 apps with dedicated modules — and the clarity of its visual canvas for orchestrating complex multi-step processes. For connecting business tools, Make is almost always the fastest path from requirement to working workflow.

The visual nature of Make has an important pedagogical benefit: it makes the data flow visible. Participants can see exactly what data enters each module, what it looks like after transformation, and what leaves for the next step. This visibility makes debugging intuitive and helps build the mental model of automation as data flowing through a pipeline of transformations.

Two sections are allocated to Make because there is a meaningful distinction between knowing how to use the interface and knowing how to design robust workflows in it. The first section covers the interface and basic workflow construction. The second covers the advanced features — routers, aggregators, iterators, error handling — that separate workflows that handle production edge cases from those that only handle the happy path.

The aggregator and iterator module pair is where Make's real power for data processing becomes apparent. An iterator takes an array and splits it into individual bundles, allowing every subsequent module to process each item independently. An aggregator collects those individual results back into a single array. Together they enable per-item processing within a single scenario — a pattern that is essential for any workflow that processes collections of records, sends personalised messages to a list of contacts, or generates individual files from a dataset.

Error handling in Make is built around the concept of incomplete runs — executions where some bundles succeeded and others failed. Make's interface shows exactly which bundles failed, what data they contained, and at which module the failure occurred. Participants learn to use this information to diagnose failures, to configure automatic retry for transient errors, and to route failed bundles to a separate error-handling branch rather than stopping the entire scenario.

- Scenarios — the fundamental unit of work in Make, containing a complete workflow
- Modules — individual steps representing connections to apps or data operations
- Connections — authenticating to external services and managing credentials
- Triggers — webhook (instant), schedule, and watch triggers and their differences
- Filters — conditional logic between modules that controls whether execution continues
- Routers — splitting execution into multiple parallel or conditional branches
- Built-in data transformation tools — string functions, date functions, number operations
- Aggregators — collecting multiple output bundles into a single structured output
- Iterators — splitting a single array into individual bundles for per-item processing
- Error handling — configuring error routes, fallback modules, and incomplete run management
- Scenario monitoring — execution history, bundle inspection, and debugging failed runs
- Scheduling — configuring run frequency and understanding Make's execution model
- Templates — using and creating reusable scenario templates
- Real use cases — CRM data sync, email workflow automation, report generation, lead routing

n8n

n8n is the platform for complex, developer-grade automation. It is open-source, which means it can be self-hosted — an important consideration for organisations with data privacy requirements that preclude sending data through a third-party cloud platform. It provides full access to JavaScript for custom logic, handles large data volumes well, and supports workflow patterns that Make cannot express — particularly complex branching, sub-workflows, and dynamic workflow construction.

The node-based interface of n8n is conceptually similar to Make's module-based interface, but the underlying philosophy is different. Make abstracts complexity. n8n exposes it. This means n8n has a steeper learning curve but fewer ceiling effects — when you need to do something complex, n8n usually has a way to do it, even if it requires writing some JavaScript.

The Code node is where n8n's real power becomes apparent. When a data transformation is too complex for n8n's built-in expression syntax — parsing a non-standard date format, applying business logic across multiple fields, generating a dynamic structure from a flat array — the Code node provides a full JavaScript runtime. Participants who completed the Python foundation find the transition to JavaScript in n8n expressions straightforward; the concepts are identical and the syntax differences are minor.

Error workflows in n8n are one of the platform's most powerful and least-used features. Rather than handling errors inline within the workflow that encounters them, n8n allows a dedicated error workflow to be triggered whenever any workflow execution fails. This separation of concerns — business logic in the main workflow, error handling in a dedicated pipeline — produces systems that are easier to maintain, easier to monitor, and more reliable in production.

- Why n8n — open source, self-hosted option, full data control, no per-operation pricing
- Running n8n — local installation with npm, Docker-based setup, cloud-hosted option
- The n8n canvas — workflow editor, node panel, execution log, and credential manager
- Node types — trigger nodes, action nodes, core nodes, and community-contributed nodes
- n8n expressions — the expression syntax for referencing previous node output and computing values
- The Code node — writing JavaScript for custom data transformation and business logic
- HTTP Request node — calling any external API from n8n with full control over request construction
- Handling complex data — working with nested objects, arrays, and mixed-type collections
- IF and Switch nodes — conditional branching with binary and multi-path routing
- Merge nodes — combining data streams from multiple workflow branches
- Loop over Items — iterating over a collection with batching support for large datasets
- Split in Batches — processing large datasets in chunks to manage memory and avoid timeouts
- Sub-workflows — extracting reusable workflow components that can be called from parent workflows
- Error workflows — dedicated error handling pipelines triggered when a workflow execution fails
- Deploying n8n — self-hosting considerations, data persistence, resource allocation, and backup
- Monitoring n8n — execution logs, failed workflow alerts, and performance observation

Key Concepts

Trigger	The event that initiates a workflow — a scheduled time, an incoming webhook, a new record, or a form submission.
Webhook	An HTTP endpoint that receives data pushed by an external system when an event occurs.

Aggregator	A Make module that collects multiple output bundles into a single structured result.
Iterator	A Make module that splits a single array into individual bundles for per-item processing.
Idempotency	The property of a workflow that can be safely re-run without duplicating effects or corrupting data.
Sub-workflow	A reusable workflow component in n8n that can be called from other workflows.
Dead Letter	A record that has failed all retry attempts and needs to be handled separately — logged, flagged, or manually reviewed.
Error Workflow	A dedicated n8n workflow triggered when any other workflow execution fails, separating error handling from business logic.
Pagination	The practice of APIs returning large datasets across multiple pages, requiring sequential calls to retrieve all records.
Webhook Signature	A cryptographic hash included in webhook request headers that verifies the request came from the genuine sender.
Polling	Repeatedly calling an API on a schedule to check for new data, as opposed to receiving a webhook when data changes.

Practice Exercises

1. Map out an automation workflow for a lead capture form that sends a confirmation email, adds the lead to a CRM, notifies the sales team on Slack, and logs the entry to a spreadsheet. Design it on paper first — triggers, actions, conditions, failure cases.
2. Build the above workflow in Make.com. Add a filter that routes leads differently based on the company size field in the form.
3. Rebuild the same workflow in n8n using the HTTP Request node to call the CRM API directly instead of using a native connector. Add a Code node to normalise the phone number format before sending.
4. Build an n8n workflow that reads a JSON file containing 100 records, processes each through an HTTP API call, handles rate limit errors with a wait-and-retry Code node, and writes successful results to a CSV.
5. Create a Make.com scenario that receives a webhook from a form submission, uses a router to branch based on the form field values, and sends different Slack messages to different channels depending on the branch.
6. Build a sub-workflow in n8n that handles the complete error notification logic for any failed workflow — it receives the error details, formats a structured Slack alert, and logs the failure to a Google Sheet. Connect it as the error workflow for three other workflows.
7. Design and implement a workflow that polls an external API every 15 minutes for new records, compares them against a list stored in Google Sheets to identify records not yet processed, processes only the new ones, and updates the tracking sheet.

MODULE 03

Generative AI Models & Tool Exploration**45**

TOTAL HOURS

The Generative AI tool landscape changes faster than any curriculum can track. New models are released monthly. Capabilities expand, prices drop, and the relative strengths of different providers shift. A curriculum that attempts to cover every tool in detail will be outdated before it is delivered.

This module takes a different approach. Rather than cataloguing tools, it builds proficiency with the three model families that define the current landscape — Claude, OpenAI, and Gemini — which between them cover every capability category in the market. A participant who understands these three families deeply will be able to evaluate any new model against what they already know and adapt quickly. ElevenLabs is added as a specialist tool for voice and audio — a capability that none of the three major families covers adequately.

The module closes with a structured tool exploration session. Rather than having participants passively receive information about new tools, this session builds the active habit of independent evaluation — identifying a new tool, assessing it against a structured framework, using it for a real task, and presenting findings to peers. This is the skill that keeps a practitioner current in a field that will not stop moving.

Claude (Anthropic)

Claude is Anthropic's model family, distinguished by its long-context capabilities, its careful instruction following, and its strength in document analysis and structured output generation. It is also the model with the most thoughtfully designed approach to extended reasoning — the Extended Thinking feature allows Claude to reason through complex problems before producing an answer, in a way that is transparent and inspectable.

The Claude family spans three tiers: Haiku for speed and cost efficiency, Sonnet for the balance of capability and performance that fits most production use cases, and Opus for the most demanding tasks where quality is more important than latency or cost. Understanding how to choose between these tiers — and when the cost of Opus is justified — is a practical skill that affects both the quality and the economics of AI systems.

- Model family overview — Haiku, Sonnet, and Opus and their capability and cost positioning
- Strengths — long-context retention, structured output generation, document analysis, instruction following
- Extended Thinking — how it works, when to enable it, latency and cost implications
- Vision capabilities — processing images, PDFs, and screenshots as model inputs
- API structure — messages format, system prompt placement, tool definitions, streaming
- Prompt caching — reducing cost and latency for prompts with repeated large context
- Constitutional AI — how Anthropic's training philosophy shapes Claude's behaviour and safety characteristics
- Practical use cases — document Q&A, structured data extraction, code review, long-form content generation
- Limitations — where Claude is less suited than alternatives and when to consider switching

OpenAI (GPT Models and Reasoning)

OpenAI's model family is the most widely deployed in the world and the de facto reference point against which other models are evaluated. GPT-4o represents the mainstream — a highly capable multimodal model that handles text, images, and audio, with strong function calling and reliable structured output. The o-series models (o1, o3) represent a fundamentally different approach to capability — models that reason before responding.

The distinction between GPT-4o and the o-series is not merely about which is 'smarter'. It is an architectural distinction. GPT-4o generates responses directly. The o-series models generate a chain of reasoning first — an internal monologue that works through the problem — and then produce a response based on that reasoning. This makes them substantially more capable on tasks that require multi-step logical reasoning, but significantly slower and more expensive.

Function calling is OpenAI's most mature tool integration feature and the one that most production AI systems are built on. Unlike prompt-based approaches to tool use where the model is asked to format a tool call as text and the caller parses it, function calling produces a structured, typed tool invocation that is unambiguous and directly executable. Structured output mode — a more recent OpenAI feature — guarantees that the model returns a response that conforms to a provided JSON schema, eliminating an entire category of parsing failures.

- Model family — GPT-4o and GPT-4o mini — capability and cost characteristics
- Multimodal inputs — text, image, and audio in a single request
- Function calling — defining tools, receiving tool call requests, returning results
- Structured output mode — guaranteed JSON adherence without prompt engineering
- o1 and o3 — reasoning models, how they differ architecturally from GPT-4o
- When to use reasoning models — tasks requiring multi-step logic, complex planning, mathematical reasoning
- Cost and latency tradeoffs — reasoning models are significantly slower and more expensive than standard models
- API structure — chat completions format, developer and system message roles, streaming
- Assistants API — file retrieval, code interpreter, and stateful conversation management
- Practical use cases — coding assistance, structured extraction, complex reasoning, customer-facing applications

Gemini (Google DeepMind)

Gemini is Google's model family and represents a genuinely different design philosophy. Where GPT-4o and Claude were primarily text models that gained multimodal capabilities over time, Gemini was designed from the ground up to be natively multimodal — treating text, image, audio, and video as first-class input types from the start.

Gemini's context window is exceptionally large — significantly larger than competing models. This changes what is possible: entire codebases, books, and long video recordings can be processed in a single call. Consider a legal document review workflow that needs to compare clauses across a 500-page contract. With a small context window, this requires chunking, embedding, retrieval, and careful prompt construction — a RAG pipeline with significant engineering overhead. With Gemini's context window, the entire document can be submitted in a single call. For these use cases, the engineering simplicity of a large context window is a genuine architectural advantage.

Video understanding is an area where Gemini currently has no serious competitor among general-purpose models. The ability to submit a video recording — a meeting, a product demo, a training session — and ask questions about its content, extract action items, or generate a structured summary opens use cases that are simply not accessible through text-only or image-only models.

- Model family — Gemini Flash (speed), Pro (balanced), and Ultra (maximum capability)
- Native multimodal design — text, image, audio, and video as first-class input types
- Industry-leading context window — implications for large document and media processing
- Gemini Thinking — extended reasoning mode and comparison with o1 and Claude Extended Thinking
- Grounding with Google Search — real-time information retrieval integrated into generation

- Google AI Studio and Vertex AI — the two deployment paths and their different audiences
- API structure — Google AI SDK, request format, streaming, and multimodal input construction
- Practical use cases — large document processing, video understanding, search-grounded generation, multimodal pipelines
- Integration with Google Workspace — Docs, Sheets, Drive, and Gmail native AI features

ElevenLabs (Voice and Audio Generation)

Voice and audio generation is the one significant capability category that the three major model families do not cover adequately. ElevenLabs has established itself as the specialist leader in this space — delivering voice quality, cloning capability, and API accessibility that significantly exceeds what general-purpose models offer for voice-specific applications.

The practical applications of voice generation in automation workflows are significant. Customer-facing applications that need audio output, content pipelines that produce podcast-format summaries, accessibility tools that convert text to natural speech, and voiceover generation for video content are all use cases where ElevenLabs provides capabilities that no general-purpose model matches.

- Text-to-speech — converting written content to natural-sounding spoken audio
- Voice library — selecting from a range of pre-built voices across languages and styles
- Voice cloning — creating a custom voice from a sample recording
- Multilingual synthesis — generating speech in different languages with natural pronunciation
- Voice design — configuring voice characteristics such as stability, clarity, and style exaggeration
- API integration — calling ElevenLabs from Python scripts and automation workflows
- Audio output formats — MP3, PCM, and streaming audio for different delivery contexts
- Latency and streaming — real-time audio generation for interactive applications
- Practical use cases — content narration, customer support audio, accessibility, video voiceover
- Cost structure — per-character pricing and how to estimate costs for production use

Model Comparison and Tool Exploration

With hands-on experience across Claude, OpenAI, Gemini, and ElevenLabs, participants are now equipped to make principled comparisons. This session formalises the comparison framework and then turns participants loose on independent exploration — building the habit of evaluating new tools as they emerge.

- Capability comparison — what each family leads on and where each has recognised limitations
- Reasoning model comparison across providers — o-series vs Extended Thinking vs Gemini Thinking
- Cost and speed comparison — matching model tier to task requirements and budget constraints
- Context window comparison — when size changes what is architecturally possible
- API surface comparison — what changes when switching providers and what remains consistent
- Multi-provider strategy — why production systems often use more than one model family
- Tool exploration framework — capability, API, use case fit, pricing, limitations, community
- Independent exploration — each participant evaluates an emerging AI tool using the framework
- Peer presentations — sharing findings, comparing evaluations, identifying disagreements
- Building the evaluation habit — staying current in a field that will not stop moving

Practice Exercises

1. Take the same complex writing task and run it through Claude Sonnet, GPT-4o, and Gemini Pro. Score each output on five criteria. Document which model performed best and why.
2. Call the ElevenLabs API from Python to convert a 500-word article into audio. Experiment with three different voices and document the differences in output quality and character.
3. Find a newly released AI tool not covered in the curriculum. Evaluate it using the five-point framework (capability, API, use case, pricing, limitations) and present your findings.
4. Design a multi-provider strategy for a content pipeline: which model for initial drafting, which for editing, which for voice-over? Justify each choice based on capability and cost.
5. Submit the same 50-page document to Claude and Gemini with the same question. Compare the quality, accuracy, and completeness of the responses. Document where each model excels.
6. Use Claude Extended Thinking and GPT-4o o1 on the same multi-step logical reasoning problem. Compare the reasoning traces, final answers, latency, and token cost.

MODULE 04

Agentic AI Frameworks**35**

TOTAL HOURS

The term 'agent' is one of the most overused in the current AI landscape. It is applied to everything from a single-turn API call with a system prompt to fully autonomous multi-step systems that plan, act, and adapt in response to what they observe. This module begins by establishing a precise definition and then builds outward from it.

A genuine AI agent is a system that perceives its environment, decides what action to take based on what it has perceived, executes that action, observes the result, and continues this loop until a goal is reached or a termination condition is met. The key properties are: the ability to take actions (not just generate text), the ability to observe the results of those actions, and the ability to use those observations to decide what to do next. A system that calls a tool once based on user input is not an agent. A system that plans a sequence of tool calls, executes them, adjusts based on intermediate results, and recovers from failures — that is an agent.

The frameworks covered in this module — LangChain, LangGraph, LlamaIndex, and MCP — represent the current production-grade stack for building agentic systems. LangChain provides the foundational building blocks. LangGraph adds stateful, graph-based orchestration for complex multi-agent workflows. LlamaIndex provides the data layer that makes agents genuinely knowledgeable. MCP is the protocol that standardises how agents communicate with tools and data sources.

Agent Fundamentals

Understanding what makes a system an agent requires understanding what makes it not one. A single LLM API call is not an agent. A fixed sequence of LLM calls where the output of one feeds into the input of the next is a chain, not an agent. What separates an agent is the combination of tool access, observation, and autonomous decision-making — the system decides what to do next based on what it has observed, rather than following a predetermined script.

The different agent architectures represent different answers to the question of how reasoning and action should be interleaved. ReAct agents reason about what to do and then do it, alternating between reasoning steps and action steps in a way the model can adjust on the fly. Plan-and-Execute agents produce a complete plan upfront and then execute each step, which works well for tasks where the steps are predictable but can be brittle when the environment does not behave as expected. Reflection agents evaluate their own outputs and revise before returning a result, which improves quality at the cost of additional latency.

Agent failure modes deserve as much attention as agent capabilities. The most common failure in production agents is the infinite loop — the agent takes an action, observes a result that does not match its goal, takes another action, and repeats indefinitely because its termination condition is never met. Implementing maximum iteration limits, detecting when the agent is repeating the same actions without progress, and designing clear termination conditions are the engineering practices that prevent runaway agents from consuming unbounded resources.

The question of when to use an agent rather than a simpler approach is one that practitioners get wrong in both directions. Some build agents for tasks that a well-designed prompt chain would handle reliably and more cheaply. Others avoid agents for tasks that genuinely require dynamic tool selection and multi-step adaptation, producing brittle prompt chains that break on every edge case. The deciding factors are whether the task requires tool use, whether the sequence of steps is dynamic rather than fixed, and whether intermediate results genuinely affect what happens next.

- LLM call vs prompt chain vs agent — the precise distinctions and why they matter

- The agent loop — perceive, think, act, observe, and repeat until goal reached
- Autonomy — what it means for a system to make decisions rather than follow a script
- Agentic failure modes — when agents get stuck, loop, hallucinate tool calls, or take wrong actions
- ReAct architecture — reasoning and acting interleaved in each loop iteration
- Plan-and-Execute architecture — full upfront planning followed by sequential execution
- Reflection architecture — agents that evaluate and revise their own output before returning
- Choosing an architecture — matching the pattern to the task and environment characteristics
- When an agent is the right tool — tasks requiring dynamic tool selection and multi-step adaptation
- When a simpler approach is better — tasks that can be solved with a well-designed prompt chain

Tool Use, Function Calling, and Memory

Tool use is the mechanism that gives agents real-world capability. Without tools, an agent can only generate text. With tools, it can search the web, query databases, read and write files, call external APIs, run code, and take actions in connected systems. The quality of an agent's tool definitions — the clarity of their descriptions, the precision of their schemas — directly determines how reliably the agent will use them correctly.

Tool definitions deserve the same care as prompts. The description field of a tool definition is not documentation for the developer — it is an instruction to the model about when to use the tool and what to expect from it. A vague tool description produces a model that calls the tool at wrong times or passes incorrect arguments. A precise tool description — specifying exactly when the tool should be used, what each parameter means, and what the return value represents — produces reliable tool use.

Memory is what gives agents continuity. The distinction between short-term and long-term memory has direct architectural consequences. Short-term memory — the conversation history in the context window — is available automatically but limited in size and lost when the context ends. Long-term memory — stored in a vector database and retrieved by semantic similarity — persists indefinitely but requires explicit write and read operations. Most production agents need both: short-term memory for the current task, long-term memory for accumulated knowledge about the user or domain that should persist across sessions.

- Function calling — the mechanism by which models request tool invocations using structured schemas
- Tool definition — name, description, and JSON schema for parameters, and why descriptions matter
- Parallel tool calling — invoking multiple tools in a single model step for efficiency
- Tool result handling — injecting tool results back into the conversation and deciding next steps
- Building custom tools — web search, file reader, database query, calculator, code executor
- Tool reliability — handling tool errors, timeouts, and unexpected return formats
- Short-term memory — the conversation history maintained within the context window
- Long-term memory — persisting information across sessions using external storage
- Episodic memory — storing records of past agent interactions and retrieving relevant ones
- Semantic memory — storing factual knowledge as embeddings for similarity-based retrieval
- Vector store integration — reading from and writing to vector databases for persistent memory
- Memory management — what to store, what to discard, and how to keep memory relevant and bounded

LangChain

LangChain is the foundational framework for building LLM-powered applications in Python. It provides the building blocks that most other frameworks build on: chains for composing LLM calls, agents for autonomous tool-using systems, memory components for conversation history, and a rich ecosystem of integrations with models, vector stores, tools, and data loaders. Understanding LangChain is prerequisite to understanding LangGraph, which extends it for stateful multi-agent workflows.

LangChain Expression Language (LCEL) is the declarative interface introduced to make chain composition more readable and more composable. Rather than constructing chains imperatively, LCEL allows chains to be built by piping components together — a style that is more concise and more amenable to parallelism and streaming. LCEL is now the recommended way to build with LangChain and is the syntax used in all new documentation.

LangSmith, LangChain's observability platform, deserves attention as a production discipline. Debugging a chain or agent that produces unexpected output is significantly harder than debugging a conventional function, because the model's reasoning is not directly inspectable from the outside. LangSmith captures the full trace of every chain and agent execution — every LLM call, every tool invocation, every intermediate output — and makes it searchable and comparable.

- LangChain's core components — models, prompts, chains, agents, memory, and tools
- Prompt templates — building parameterised prompts with LangChain's template classes
- LLM and Chat Model wrappers — a unified interface for calling models from different providers
- LangChain Expression Language (LCEL) — composing components with the pipe operator
- Chains — sequential compositions of components where each step's output feeds the next
- Memory components — ConversationBufferMemory, ConversationSummaryMemory, and vector store memory
- Tool integration — defining tools from functions, APIs, and LangChain's built-in tool library
- Agents — the agent executor, ReAct agent, and OpenAI functions agent patterns
- Callbacks and streaming — observing intermediate steps and streaming output to users
- LangSmith integration — tracing and evaluating chains and agents for debugging and optimisation
- Practical patterns — when LangChain adds value and when it introduces unnecessary complexity

LangGraph

LangGraph extends LangChain to handle stateful, graph-based workflows. The motivating insight is that real-world agent behaviour is not a straight line — it involves cycles, branches, backtracking, and coordination between multiple specialised agents. LangChain's linear chains cannot naturally express these patterns. LangGraph models agent behaviour as a directed graph where nodes are processing steps, edges define transitions, and a shared state object persists across all steps.

The state-centric design of LangGraph is its key innovation. Every node reads from and writes to a shared state dictionary. This makes the full context of the workflow visible at every step, makes debugging tractable, and enables persistence — the state can be serialised and stored, allowing workflows to be paused, resumed, and even handed off between different processes.

The supervisor pattern deserves particular attention because it maps directly to how complex real-world tasks are organised. A supervisor agent receives a high-level goal, breaks it into subtasks, routes each subtask to the specialised worker agent best equipped to handle it, collects and synthesises the results, and returns a coherent answer. The workers know nothing about each other — they only communicate through the supervisor and the shared state. This architecture is both powerful and testable; each worker can be evaluated independently before the full system is assembled.

Human-in-the-loop patterns in LangGraph address one of the most important practical questions in production AI deployment: when should a human review what the agent is about to do before it does it? LangGraph's interrupt mechanism allows workflows to pause at any node, surface the current state to a human reviewer, and resume only after approval is received. This is not a workaround for unreliable agents — it is a deliberate architectural choice for high-stakes workflows where human oversight is required regardless of agent capability.

- Why LangGraph — the limitations of linear chains for cycles, branching, and multi-agent coordination
- Nodes — processing units in the graph, each implementing a specific step in the workflow

- Edges — transitions between nodes, either unconditional or conditional based on state
- State — the shared dictionary that persists and evolves across all nodes in the graph
- State schemas — defining the shape and types of the state with Python TypedDict
- Conditional edges — branching the graph based on the current state or a model decision
- Cycles — loops in the graph that allow agents to retry, reflect, and iterate
- The END node — the termination condition that stops the graph execution
- Human-in-the-loop — inserting checkpoints where human review or approval is required
- Supervisor pattern — an orchestrator agent that routes tasks to specialised worker agents
- Parallel execution — running multiple graph branches simultaneously for efficiency
- Persistence — serialising graph state to storage for long-running or resumable workflows
- Building a complete multi-agent workflow with LangGraph from requirements to running code

LlamaIndex, MCP, and Building from Scratch

LlamaIndex specialises in the data layer of agentic systems. Where LangChain and LangGraph focus on orchestration — how agents reason, plan, and coordinate — LlamaIndex focuses on how agents access and understand knowledge. Its data ingestion, indexing, and query capabilities make it the right choice for any agent that needs to work with large document collections, structured data sources, or complex retrieval requirements.

The Model Context Protocol (MCP) addresses a different problem: standardisation. Every agentic framework has historically defined its own tool interface. MCP defines a common protocol — a shared language for how agents describe what tools they need and how tool servers describe what they offer — that eliminates this fragmentation. A tool server built once can be used by any MCP-compatible agent, in any framework, on any platform.

Building a custom MCP server — exposing an internal API, a database, or a proprietary data source via the MCP protocol — is covered in the practical component of this section. Participants implement a working MCP server in Python, connect it to a LangGraph agent, and verify that the agent can discover and use the exposed tools without any framework-specific integration code. This is the practical proof of why MCP matters.

- LlamaIndex data loaders — ingesting documents from files, databases, APIs, and cloud storage
- Node parsers — chunking strategies for different document types and retrieval requirements
- Index types — VectorStoreIndex, KeywordTableIndex, and TreeIndex
- Query engines — natural language interfaces over indexed data
- Retrieval modes — top-k, hybrid, and reranked retrieval strategies
- Response synthesisers — combining retrieved content into coherent answers
- LlamaIndex agents — combining retrieval query engines with tool-use loops
- Connecting LlamaIndex retrieval to LangGraph agent workflows
- MCP architecture — clients, servers, and the protocol specification
- MCP server types — tool servers, resource servers, and prompt servers
- Connecting agents to MCP servers from LangChain and LangGraph
- Building a custom MCP server — exposing a database or API via the protocol
- Workshop — building a complete agent with LangGraph orchestration, LlamaIndex data access, and MCP tools

Key Concepts	
Agent Loop	The cycle of perceive, think, act, observe, and repeat that defines agent behaviour.
Function Calling	The mechanism by which a model requests the invocation of a defined tool using a structured schema.

LangGraph State	A shared dictionary that persists across all nodes in a LangGraph workflow, carrying the full context of execution.
MCP	Model Context Protocol — an open standard for how agents communicate with tools and data sources.
Supervisor Pattern	A multi-agent architecture where an orchestrator agent routes tasks to specialised worker agents.
ReAct	Reasoning and Acting — an agent architecture where the model alternates between reasoning steps and action steps.
LlamaIndex	A framework specialising in data ingestion, indexing, and retrieval for AI agent knowledge access.
Human-in-the-Loop	An architectural pattern where agent workflows pause for human review or approval before proceeding.
Episodic Memory	Storage of records of past agent interactions, retrievable to inform future behaviour.
LCEL	LangChain Expression Language — a declarative syntax for composing LangChain components using the pipe operator.
Conditional Edge	In LangGraph, an edge that routes to different nodes based on the current state or a model decision.
LangSmith	LangChain's observability platform for tracing, debugging, and monitoring chain and agent executions.

Practice Exercises

1. Build a ReAct agent with LangChain that has three tools: web search, calculator, and a file writer. Give it a research task and observe how it decides which tools to use and in what order.
2. Implement the same agent as a LangGraph workflow with explicit state management. Add a reflection node that evaluates the draft answer before returning it.
3. Use LlamaIndex to ingest a collection of 20 PDF documents, build a VectorStoreIndex, and connect a query engine as a tool in a LangGraph agent. Ask the agent questions that require synthesising information across multiple documents.
4. Build a custom MCP server in Python that exposes a SQLite database as a tool. Connect a LangChain agent to it and verify that the agent can query the database as part of answering questions.
5. Implement a supervisor-worker multi-agent system in LangGraph: a supervisor that receives a research question, routes it to a search worker and a summarisation worker in parallel, and synthesises their outputs into a final answer.
6. Add a human-in-the-loop checkpoint to a LangGraph workflow that pauses before executing a write operation, displays the proposed action to the user, and proceeds only after explicit approval.
7. Build a LangGraph agent with both short-term memory (conversation buffer) and long-term memory (vector store). Test it across multiple separate runs and verify that it can recall facts from previous sessions.

MODULE 05

Python for Generative AI — Integration and Implementation

45

TOTAL HOURS

Python is the dominant language of the AI engineering ecosystem. Every major AI model provides a Python SDK. Every agentic framework is written in Python. Every automation platform with a developer API supports Python integrations. This module is structured in two parts. Part A covers the Python scripting foundation — the specific subset of Python that appears in Generative AI work, from API communication and SDK usage through to batch processing and chaining. Part B is the integration capstone — embedding Generative AI capabilities inside the automation workflows built in earlier modules, with a focus on reliability, cost management, and production readiness.

By this point in the program, participants have worked through four modules of hands-on exposure to APIs, frameworks, and tools. Much of the Python foundation has been absorbed implicitly. Part A therefore focuses on the patterns that matter most for AI engineering rather than language fundamentals, building the scripting depth needed to work confidently at the integration layer. Part B then brings every preceding module together in complete, documented, deployed AI-automation systems.

Python for Generative AI

Core Python Fundamentals for AI Work

The starting point is not Hello World. It is the specific subset of Python that appears constantly in AI integration code — data structures, control flow, and functions in the contexts where AI work actually uses them. This means strings and how to manipulate them (because every prompt is a string), dictionaries and how to navigate them (because every API response is a dictionary), and functions and how to write them (because reusable prompt templates are functions).

Error handling receives particular attention here. AI API calls fail. Networks drop. Rate limits are hit. A script that crashes on the first error is not a useful script. Participants learn to write code that handles failures gracefully — logging what went wrong, retrying where appropriate, and failing cleanly when retrying is not possible.

- Strings, integers, booleans — data types in the context of AI inputs and outputs
- Lists and dictionaries — the structures that carry AI data through a pipeline
- Control flow — if-else, for loops, while loops as they appear in AI scripts
- Functions — writing reusable prompt templates and API call wrappers
- Reading from and writing to files — text, JSON, and CSV as AI pipeline data sources
- Error handling with try-except — managing failures in API-dependent code
- Debugging — reading Python tracebacks, identifying failure points, print debugging
- Code organisation — structuring a script so it is readable and modifiable

Working with APIs and JSON

Every AI model in this curriculum is accessed via a REST API. Every response comes back as JSON. Understanding how these two things work — HTTP communication and JSON data structures — is the difference between someone who can only use pre-built tools and someone who can connect to any AI service that publishes an API.

The requests library is the primary tool here. It is the most widely used Python library for HTTP communication and the one participants will encounter in virtually every AI integration codebase. The focus is on the patterns that appear in AI API usage specifically: POST requests with JSON bodies, authentication via headers, handling paginated responses, and building retry logic for rate-limited endpoints.

Authentication is treated as a security topic from the beginning, not as a convenience. The pattern of placing an API key in an Authorization header is simple, but the discipline of never exposing that key — not in code, not in logs, not in error messages printed to the console — is a professional standard that participants establish as a habit in this module. Every code example shows credentials loaded from the environment, never hardcoded, because the examples set the standard for the code that participants will write after the program.

Rate limiting is addressed directly because every AI API imposes limits, and a script that does not handle them will fail in production. The pattern of detecting a 429 status code, waiting for the duration specified in the Retry-After header, and retrying the request is implemented as a reusable utility function that participants carry into every subsequent piece of work. Exponential backoff — doubling the wait time with each retry up to a maximum — is introduced as the production-standard approach.

- REST API fundamentals — resources, HTTP methods, endpoints, and the request-response cycle
- Making HTTP requests with the requests library — GET and POST in practice
- Request structure — base URL, path parameters, query strings, headers, and request body
- Authentication patterns — API keys in headers, bearer tokens, and how to pass them safely
- JSON structure — objects, arrays, nested data, and navigating multi-level responses
- Parsing API responses — extracting content from model response objects
- HTTP status codes — understanding what 200, 400, 401, 429, and 500 mean and how to handle each
- Rate limiting — identifying rate limit errors and implementing exponential backoff
- Pagination — handling APIs that return large datasets across multiple calls

Environment Management and AI SDKs

Credential management is not a security lecture — it is a practical skill that affects every person who writes code that calls an API. API keys committed to version control are a consistently common and costly mistake. This session treats secrets management as a first-class topic and establishes habits that participants carry into every project.

The Anthropic and OpenAI Python SDKs are covered in parallel, because their patterns are similar and understanding one makes the other immediately familiar. Both follow the same basic structure: create a client, pass a list of messages, receive a response object. Reading the full response object — not just the generated text — is a habit that separates careful practitioners from careless ones. The response object contains the finish reason, the usage statistics, and metadata that can be used to detect anomalies. Streaming responses change the user experience of AI applications fundamentally — implementing streaming correctly in Python requires understanding asynchronous iteration, handling the stream object, and reassembling the full response from the token stream.

- Why hardcoding API keys in source code is dangerous — how credentials leak via version control
- Using .env files — storing credentials outside of code in key-value format
- Loading environment variables with python-dotenv — the standard pattern for AI projects
- Accessing variables at runtime with os.environ — safe credential retrieval in scripts
- Managing multiple environments — different API keys for local, staging, and production
- Anthropic SDK — client instantiation, the messages API, system prompts, and response objects
- OpenAI SDK — client instantiation, chat completions, roles and message structure
- Request parameters — model selection, temperature, max tokens, stop sequences
- Parsing response objects — extracting text, checking finish reason, reading usage data

- Streaming responses — receiving and processing tokens as they arrive
- Token counting — estimating and measuring usage for cost management
- Handling SDK-specific errors — `AuthenticationError`, `RateLimitError`, `APIConnectionError`

Building AI Scripts

This session moves from individual API calls to complete scripts that do something useful. Each pattern introduced here maps directly to something that appears in production AI systems — batch processors, conversation managers, content pipelines, and data extractors. Participants write these patterns themselves rather than running example code.

Logging and cost tracking deserve specific attention. In a development environment, printing to the console is sufficient. In a production environment, structured logging that records what was sent, what was received, how many tokens were consumed, and whether the call succeeded is not optional. Participants build logging into their scripts from the start rather than retrofitting it after things go wrong.

The final pattern — chaining API calls — is where the real power of scripted AI work becomes apparent. A single model call is limited in what it can do well. A sequence of calls — where the first extracts structure, the second enriches it, and the third formats the result — can produce outputs that no single prompt reliably achieves. Understanding how to design these chains, manage the state between calls, and handle failures at any point in the sequence is a foundational production skill.

- Single-turn scripts — one prompt, one response, structured output written to file
- Multi-turn conversation scripts — maintaining message history across multiple API calls
- Batch processing — reading a list of inputs from CSV or JSON and processing each through the model
- Chaining calls — using the output of one model call as the input to the next
- Parameterised prompts — building prompt templates with dynamic variable substitution
- Writing results — JSON output files, CSV exports, and plain text reports
- Logging — recording prompts, responses, timestamps, and token usage for auditability
- Progress tracking — showing progress for long-running batch jobs
- Cost tracking — accumulating token usage and calculating cost across a batch run

Generative AI Meets Automation

Integration Architecture

The integration layer is where Generative AI capabilities live inside automation workflows. Before building anything, it is worth understanding the design decisions this layer involves — because the wrong decision at the architecture level produces workflows that are expensive, unreliable, or impossible to maintain.

Cost is a particular concern. An automation workflow that runs a thousand times a day and makes an AI API call on each run is generating a thousand AI API calls per day. At GPT-4o pricing, that can be thousands of dollars per month. At Gemini Flash or Haiku pricing, it might be a fraction of that. Choosing the right model for the right step — not always the most capable one, but the most appropriate one — is an architectural decision with significant economic consequences.

Output contracts are the integration detail that most teams skip and then regret. When an AI call sits inside a workflow, the downstream steps depend on what the AI returns. If the model is asked to return JSON and instead returns a markdown code block containing JSON, the downstream parser fails. If the field names in the JSON change between prompt versions, every downstream reference breaks silently. Defining an explicit output contract — specifying exactly what the AI must return, in what format, with what field names — and enforcing it

with a validation step in the workflow before passing data downstream is the pattern that prevents this entire class of failure.

Prompt stability in production is a discipline that mirrors software versioning. When a prompt is changed — even a small change intended to improve one case — the change should be tested against the full range of inputs the workflow receives before deployment. A prompt that works better for the case that motivated the change but breaks three edge cases that were previously handled correctly is not an improvement. Participants learn to treat prompt changes as code deployments: version-controlled, tested, and rolled out deliberately rather than edited in place.

- Where in the workflow the AI call belongs — trigger, mid-process, or output-generation step
- Synchronous integration — the workflow waits for the AI response before continuing
- Asynchronous integration — the AI call is dispatched and the result is collected when available
- Latency management — AI calls are 1-30 seconds, not milliseconds — design for this difference
- Cost management — per-call pricing means high-volume workflows need careful model selection
- Output contracts — defining exactly what the AI will return so downstream steps can rely on it
- Prompt stability — changes to prompts can break downstream parsing; treat prompts like code
- Failure modes specific to AI calls — timeout, context overflow, unexpected format, rate limit
- Fallback strategies — what the workflow does when the AI call fails after retries
- Caching — avoiding redundant AI calls for identical or near-identical inputs
- Choosing the right model for each step — capability vs cost vs latency trade-offs in context

Generative AI in Make.com

Make's HTTP module is the primary mechanism for calling AI APIs from a Make workflow. Any AI model with a REST API can be called from Make — Claude, GPT-4o, Gemini, ElevenLabs — using the same HTTP module with different configurations. This session covers the practical patterns for making these calls reliably, parsing the responses correctly, and routing the results appropriately to downstream workflow steps.

- Calling AI APIs via the HTTP module — method, URL, headers, and body configuration
- Constructing dynamic prompts from workflow data — injecting customer names, document content, previous step outputs
- Authentication in the HTTP module — passing API keys in headers securely using connections
- Parsing AI JSON responses in Make — extracting text, metadata, and structured fields
- Handling streaming vs non-streaming responses in Make's HTTP module
- Conditional routing based on AI output — different workflow branches for different model responses
- Error routes for failed AI calls — fallback behaviour, Slack alerts, and failure logging
- Using Make's native OpenAI module — when it helps and when the HTTP module gives more control
- Rate limit handling in Make — delays between AI calls and retry configuration
- End-to-end build — email triage workflow: classify incoming emails with Claude, route to teams, draft replies

Generative AI in n8n

n8n provides multiple paths for integrating AI: the AI Agent node for LangChain-based agents, the HTTP Request node for direct API calls, and community nodes for specific providers. For complex AI integration — chaining multiple AI calls, transforming responses with custom logic, building conditional AI pipelines — n8n provides more control and flexibility than Make.

The combination of n8n's AI Agent node and the Code node creates a particularly powerful pattern for document processing pipelines. The AI Agent node handles the intelligence — understanding the document structure, extracting relevant fields, classifying content. The Code node handles the engineering — validating the extracted

fields, transforming them into the exact format the destination system expects, and flagging records that fall outside the expected range for human review. This separation of concerns keeps the AI prompt focused on what AI does well and keeps the data engineering in code where it is testable and maintainable.

Rate limit management in n8n requires explicit handling because n8n does not throttle API calls automatically. A loop that iterates over 500 records and makes an AI API call for each will fire all 500 requests as fast as the network allows — which will trigger rate limit errors after the first few dozen. Implementing a wait step between calls, batching records into groups, or using a queue-based pattern are all strategies covered here.

- AI Agent node — n8n's built-in LangChain integration for deploying agents in workflows
- Configuring the AI Agent node — model selection, memory, tool connections, and system prompt
- HTTP Request node for AI APIs — full control over request construction and response parsing
- Calling Claude, GPT-4o, and Gemini via HTTP Request node with authentication
- Code node for AI response processing — parsing, validating, and transforming model outputs
- Chaining AI calls — passing one model's output as input to the next step or model
- Looping with AI — running an AI call over each item in a dataset with batch size control
- Handling AI output variability — using Code node to normalise inconsistent model responses
- Rate limit handling — implementing wait-and-retry logic in n8n with the Code node
- End-to-end build — document processing pipeline: ingest PDFs, extract structured fields with AI, load to database

End-to-End Use Case Walkthroughs

Each use case in this session is walked through from a plain-language requirement to a working, deployed system. Participants observe the full design process — tool selection, workflow architecture, prompt design, error handling, and documentation — before building their own variation of the same use case.

The content pipeline use case is deceptively instructive. On the surface it appears simple — generate content, review it, publish it. In practice it surfaces every significant integration challenge: the AI call must return content in a consistent format, the human review step must integrate with a real approval mechanism, the publishing step must handle the target platform's API, and the entire pipeline must handle failures at any stage gracefully.

The data enrichment pipeline use case represents the highest-volume AI automation pattern in enterprise environments. Incoming records are passed through an AI model for classification, entity extraction, sentiment scoring, or summarisation, and the results are written back to the source system or a downstream database. The design challenges here are primarily operational: managing throughput, handling the percentage of records where the AI returns unexpected output, and ensuring the system recovers cleanly from interruptions without processing records twice or missing records entirely.

- Content pipeline — AI drafts content, automation routes it for human review, publishes on approval
- Customer support automation — AI classifies tickets by category and urgency, routes to queue, drafts first response
- Research assistant — automation retrieves sources, AI summarises and extracts key points, report is generated and sent
- Data enrichment — incoming records are passed to AI for classification or entity extraction, results written back to source
- Invoice processing — documents ingested, AI extracts line items and totals, results validated and loaded to accounting system
- Meeting intelligence — transcripts ingested, AI extracts action items and decisions, results distributed to stakeholders

System Design, Documentation, and Production Readiness

The final section puts participants in the role of system designer and hands them a requirement they have not seen before. Working from scratch, they design the architecture, choose the tools, implement the workflow, handle the failure cases, document the system, and deploy it. This is the practical proof of everything learned across the five modules.

System design for AI-automation systems differs from conventional software design in one important respect: the AI components are non-deterministic. Two identical inputs to the same model with the same temperature setting can produce different outputs. This means that the correctness guarantees available in conventional software do not apply. System design must account for this: validation steps that catch outputs outside the expected range, human review checkpoints for high-stakes decisions, and monitoring that detects when output quality degrades over time as models are updated or prompts drift.

Documentation for AI-automation systems has components that conventional software documentation does not require. In addition to the standard architecture diagrams and data flow documentation, an AI-automation system requires: the prompts used at each AI step and the reasoning behind their design, the expected output format and the validation logic that enforces it, the model version in use and the date of last evaluation, and the criteria under which a prompt or model should be reviewed.

- System design process — from requirement to architecture diagram to implementation plan
- Tool selection decisions — when to use Make vs n8n, which AI model for which step, when agents are needed
- Documenting AI-automation systems — architecture, data flows, AI integration points, failure handling
- Monitoring in production — metrics to watch, thresholds to alert on, and how to detect pipeline degradation
- Prompt maintenance — detecting and responding to prompt drift as model behaviour evolves
- Handling upstream changes — what breaks when connected APIs or data formats change
- Handover — what a system handover document contains and why it matters
- Final project — participants build a complete AI-automation system from a provided requirement specification

Key Concepts	
Token	The basic unit of text that a language model processes. Roughly 3/4 of a word on average in English.
SDK	Software Development Kit — a library that wraps an API and provides a convenient interface for a specific programming language.
Environment Variable	A value stored in the operating environment rather than in code, used to pass configuration and credentials safely.
Streaming	Receiving model output token by token as it is generated, rather than waiting for the complete response.
Exponential Backoff	A retry strategy that doubles the wait time between each retry attempt, preventing overwhelming a rate-limited API.
Finish Reason	A field in the model response indicating whether generation stopped naturally or was cut short by a token limit.
Batch Processing	Running the same operation across a collection of inputs, typically by reading from a file and writing results back to a file.
Output Contract	An explicit specification of what an AI call must return — format, field names, and types — that downstream steps depend on.

Prompt Stability	The discipline of treating prompt changes as code deployments — version-controlled, tested, and rolled out deliberately.
Prompt Drift	The gradual degradation of prompt performance over time as underlying model behaviour changes with updates.

Part A — Python Exercises

Practice Exercises

1. Write a Python script that reads a list of product names from a CSV file, calls Claude to generate a one-sentence description for each, and writes the results back to a new CSV with a description column.
2. Extend the above script with retry logic — if an API call fails due to a rate limit error, wait 30 seconds and try again, up to 3 times before logging the failure and moving on.
3. Build a multi-turn conversation script that maintains context across 5 exchanges with a model and saves the full conversation history as a JSON file at the end.
4. Write a parameterised prompt template function that takes a customer name, product category, and issue description, and produces a structured support response using the OpenAI SDK.
5. Build a script that calls the same prompt across Claude, GPT-4o, and Gemini, logs the response from each including token usage and latency, and writes a comparison CSV.
6. Implement a streaming response handler that prints tokens to the console as they arrive and accumulates them into a final string, handling the stream correctly for both the Anthropic and OpenAI SDKs.

Part B — Integration Exercises

Practice Exercises

1. Build a complete email-to-task workflow in Make: incoming emails are classified by Claude as either support, sales, or internal. Each category routes to a different project management tool and creates a task with AI-generated title and description.
2. Build a document intelligence pipeline in n8n: PDFs are dropped into a folder, n8n picks them up, sends each page to GPT-4o for structured data extraction, validates the output with a Code node, and writes the results to a Google Sheet.
3. Design a production-ready AI-automation system for a use case of your choice. Document the architecture, the data flows, the AI integration points, the failure handling strategy, and the monitoring plan. Present it to the group.
4. Build an output contract validation step in n8n: after each AI API call, a Code node checks that the response is valid JSON, that all required fields are present and of the correct type, and routes to an error branch if validation fails.
5. Implement a cost-aware workflow that calls a cheap fast model first for initial classification, and only escalates to an expensive model when the cheap model returns a low-confidence result. Measure and compare cost and accuracy across 50 test inputs.
6. Build a content approval pipeline end to end: an AI drafts content based on a brief, the draft is posted to a Slack channel with approve and reject buttons, and the workflow publishes to a CMS on approval or logs the rejection with feedback.

Program Completion

Participants completing this program will be able to:

- Write Python scripts to call, chain, and orchestrate AI model APIs across providers
- Explain how Large Language Models work — tokenization, attention, RAG, fine-tuning — at a conceptual level sufficient to make good engineering decisions
- Work confidently across Claude, OpenAI, and Gemini model families and select the right model for a given task
- Understand when and why reasoning models change what is architecturally possible
- Evaluate new AI tools independently using a structured framework
- Use ElevenLabs for voice and audio generation integrated into content pipelines
- Design and build automation workflows using Make.com and n8n
- Connect any system with a REST API to an automation workflow
- Build AI agents and multi-agent workflows using LangChain, LangGraph, LlamaIndex, and MCP
- Integrate Generative AI capabilities inside production automation pipelines
- Document, monitor, and maintain the AI-automation systems they build
- Design complete AI-automation architectures from requirements to deployment

Tools Covered

Category	Tool / Technology
Language & SDKs	Python, Anthropic SDK, OpenAI SDK
LLM Models	Claude (Haiku/Sonnet/Opus), GPT-4o, GPT-4o mini, o1, o3, Gemini Flash/Pro/Ultra
Voice & Audio	ElevenLabs
Automation Platforms	Make.com, n8n
Agent Framework & Orchestration	LangChain, LangGraph
RAG & Data Pipelines	LlamaIndex
Agent Protocol	MCP (Model Context Protocol)